

Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 7 & 8: The UNIX Interface and System Calls

Author:
st. gr. FAF-241 Mitrofan Alexia
Verified:
asist. univ. Reițman Patricia

Chișinău - 2026

Contents

OBJECTIVES **2**

LABORATORY 7 **3**

Before the start 3

Part 1. fork, exec, wait 4

Part 2. A zombie, on purpose 7

Part 3. Redirection: ls -l > out.txt 9

Part 4. ls /etc — wc -l 11

LABORATORY 8 **14**

Before the start 14

Part 1. run, inside xv6 14

Part 2. pingpong 20

CONCLUSION **23**

REFERENCES **24**

OBJECTIVES

The goal of these two labs was to see in practice how an operating system creates processes and connects them with each other, first on Linux and then inside the small xv6 kernel. In detail, the objectives were:

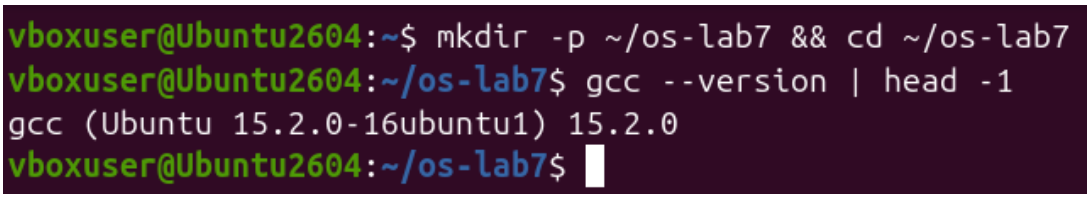
- to understand how a process is created and how a shell runs a command, using the three system calls `fork`, `exec` and `wait`, and to see what happens when `exec` fails;
- to observe the life cycle of a process, including a zombie: a child that has terminated but has not yet been reaped by its parent;
- to understand file descriptors and how a shell implements redirection (`ls -l > out.txt`) with `close` and `open`, without changing the program that is run;
- to build by hand a pipeline equivalent to `ls /etc | wc -l` using `pipe`, `fork`, `dup` and `close`, and to understand why every unused end of the pipe must be closed;
- to run the same calls inside xv6 and compare them with their Linux counterparts (`execlp` and `waitpid` versus `exec` and `wait`);
- to implement `pingpong`, a two-way communication between a parent and a child through two pipes, and to understand how blocking `read` synchronizes the processes.

LABORATORY 7

Before the start

Before starting this lab, we check our C and C++ compiler's latest version, since we need it.

```
mkdir -p ~/os-lab7 && cd ~/os-lab7
gcc --version | head -1
```



Creating lab 7 folder and checking the compiler's version

Part 1. fork, exec, wait

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();  /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {  /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(),
            getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls
            */
        perror("exec");  /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);  /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(
        status));
    return 0;
}
```

```
vboxuser@Ubuntu2604:~/os-lab7$ touch run.c
vboxuser@Ubuntu2604:~/os-lab7$ gedit run.c
```

Figure 1.1: Creating and editing run.c

Open ~/os-lab7 Save — run.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                         /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                             /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                         /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 1.2: run.c

```
gcc -Wall -Wextra -o run run.c && ./run
```

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4430
child:  my PID is 4432, my parent is 4430
total 24
-rwxrwxr-x 1 vboxuser vboxuser 16392 Oct 22 22:46 run
-rw-rw-r-- 1 vboxuser vboxuser   914 Oct 22 22:44 run.c
parent: child 4432 ended with status 0
```

Figure 1.3: Compiling run.c (ls)

```
execlp("date", "date", (char *)NULL);
```

```c
if (pid == 0) {                             /* only the child runs this */
    printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
    fflush(stdout);
    execlp("date", "date", (char *)NULL);
    perror("exec");                         /* runs only if exec failed */
    exit(127);
}
```

Figure 1.4: Changing run.c with the specified line

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4675
child:  my PID is 4676, my parent is 4675
Thu Oct 22 10:47:36 PM UTC 2026
parent: child 4676 ended with status 0
```

Figure 1.5: Results of modified run.c (date)

```
1 execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 5055
child:  my PID is 5056, my parent is 5055
exec: No such file or directory
parent: child 5056 ended with status 127
```

Figure 1.6: Results of invalid run.c (nosuchprog)

1. **The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`?**
   A successful `execlp` replaces the child's whole program with the new one, so the code after it (including `perror`) no longer exists and never runs. `execlp` returns only on failure. With `ls` and `date` the exec succeeds; with `nosuchprog` no such executable is found in `$PATH`, so exec fails, returns, and `perror` prints ``No such file or directory''.

2. **Which number does the child print as "my parent", and whose PID is it?**
   The child prints `getppid()`, which is the PID of the parent process, aka the same value as the parent's PID (in our cases: 4430, 4675, 5055). It is the process that called `fork()`.

3. **What status does the parent see with `ls`, and what status with `nosuchprog`?**
   With `ls`, the parent sees 0, which means that the command executed succesfully, or more accurate: the child exited successfully. With `nosuchprog`, it sees 127, which can mean that either the command we just executed isn't a built-in one, or the command isn't in `$PATH`. In our case, the command isn't in `$PATH`, so the child continued to `exit(127)`, following the shell convention for ``command not found''.

Part 2. A zombie, on purpose

```
int status;
sleep(20);  /* the new line */
waitpid(pid, &status, 0); /* step 3: wait for the child */
```

```
vboxuser@Ubuntu2604:~/os-lab7$ touch zombie.c
vboxuser@Ubuntu2604:~/os-lab7$ gedit zombie.c
```

Figure 2.1: Creating and editing zombie.c

Open — zombie.c — ~/os-lab7 — Sav

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    sleep(20);                                   /* the new line */
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 2.2: zombie.c

```
gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2; ps -o pid,ppid,stat,cmd
```

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2; ps -o pid,ppid,stat,cmd
[1] 5321
    PID    PPID STAT CMD
   4389    4352 Ss   /usr/bin/bash
   5321    4389 S    ./zombie
   5324    5321 Z    [ls] <defunct>
   5329    4389 R+   ps -o pid,ppid,stat,cmd
```

Figure 2.3: Compliling and results of zombie.c

```
1 ps -o pid,ppid,stat,cmd
```

```
vboxuser@Ubuntu2604:~/os-lab7$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   4389    4352 Ss   /usr/bin/bash
   5339    4389 R+   ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
```

Figure 2.4: Results of zombie.c after waiting 20 seconds

1. **Which letter does `ls` have in the STAT column, and what does it mean?**
   In `STAT`, `ls` has the letter Z, which stands for *zombie*, which means that the process has already been terminated byt the parent has not yet collected its exit status with `wait`/`waitpid`, so its entry remains in the process table (shown as `<defunct>`). Here the parent `./zombie` was still in `sleep(20)`, so it had not reaped the finished child.

2. **After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear?**
   No, the line is not there. It is line 25, `waitpid(pid, &status, 0)`, which made it disappear. Once the parent woke up from `sleep(20)`, `waitpid` reaped the child and the kernel removed its process-table entry. The `sleep(20)` on line 24 only delayed this, which is what made the zombie visible in the first place.

Part 3. Redirection: ls -l > out.txt

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);  /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as
            always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

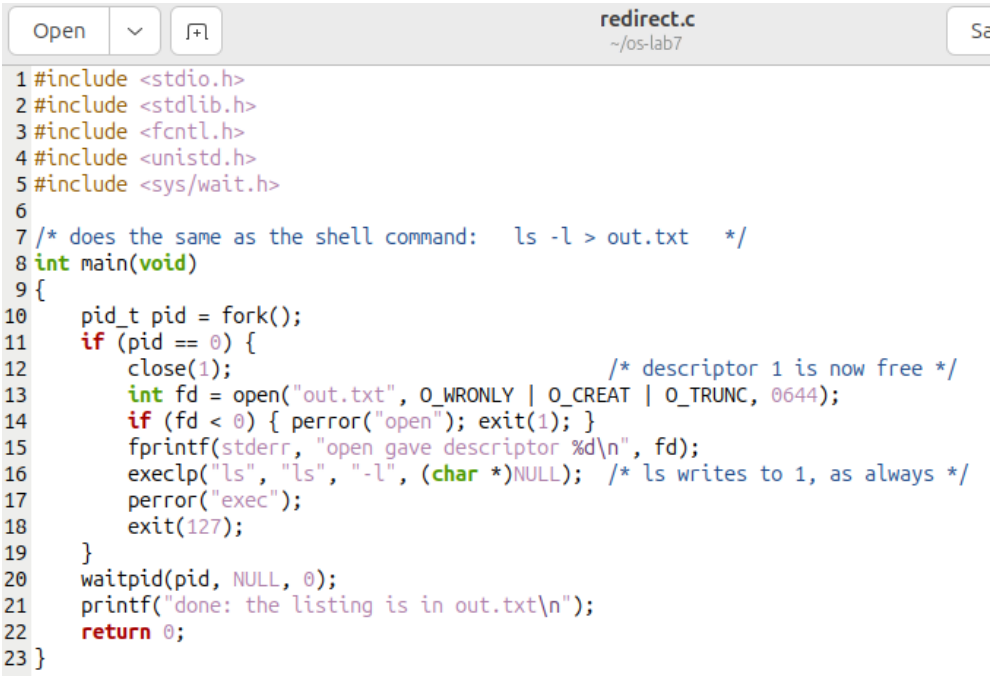


Figure 3.1: redirect.c

```
gcc -Wall -Wextra -o redirect redirect.c && ./redirect
head -3 out.txt
```

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
head -3 out.txt
open gave descriptor 1
done: the listing is in out.txt
total 68
-rw-r--r-- 1 vboxuser vboxuser     0 Oct 22 22:52 out.txt
-rwxrwxr-x 1 vboxuser vboxuser 16336 Oct 22 22:52 redirect
```

Figure 3.2: Compiling and results of redirect.c

1. **Which number did `open` give, and why exactly that one?**
   `open` gave the number 1. `open` always returns the lowest free file descriptor, and since line 12 closed descriptor 1 (0 and 2 were still in use), 1 was the lowest free one.

2. **`ls` was not changed at all. Why does its listing not appear on the screen?**
   After line 12, `close(1)`, the `open` call gives `out.txt` descriptor 1, and this descriptor stays open across `execlp`. `ls` still writes to descriptor 1 as always, but descriptor 1 now points to `out.txt`, so the listing goes into the file instead of the screen.

3. **To redirect the errors instead (like `2 > err.txt`), which descriptor would you close before open?**
   I would close descriptor 2, which is `stderr`. Then `open` would return 2, and everything written to standard error would go into the file.

Part 4. ls /etc | wc -l

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {  /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);  /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {  /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);  /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);  /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Open | pipeline.c ~/os-lab7 | Save

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Figure 4.1: pipeline.c

```
gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
ls /etc | wc -l
```

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
ls /etc | wc -l
238
238
vboxuser@Ubuntu2604:~/os-lab7$
```

Figure 4.2: Compiling and results of pipeline.c

```
close(p[0]);  /* the parent uses neither end */
/* close(p[1]); */
```

```
 close(p[0]);        /* the parent uses neither end */
/* close(p[1]); */
 wait(NULL);
 wait(NULL);
 return 0;
```

Figure 4.3: Commenting out the specified lines

```
vboxuser@Ubuntu2604:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
```

Figure 4.4: Results of modified pipeline.c

1. **Did `./pipeline` and `ls /etc | wc -l` give the same number?**
   Yes, they gave the same number. It is expected because `pipeline.c` builds exactly the same pipeline that the shell builds for `ls /etc | wc -l`.

2. **After the change, which of the two children never ends, `ls` or `wc`?**
   `wc`, because we comment out `close(p[1])`, which is the pipe responsible for writing the results on screen. More explicitly, `close(p[1])` is in the parent, which means that the parent keeps a copy of the write end of the pipe open. `ls` ends normally, but `wc` never ends (and therefore the parent's second `wait` blocks too).

3. **Why, in one sentence: what is `wc` waiting for?**
   `wc` is waiting for end-of-file on the pipe, which only arrives when all copies of the write end are closed, but the parent still holds one open.

LABORATORY 8

Before the start

```
vboxuser@Ubuntu2604:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -glo
bal virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk
-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

Verifying if xv6 builds

Part 1. run, inside xv6

```c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();  // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){  // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);  // step 2: become echo
    printf("exec failed\n");  // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);  // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Open

run.c
~/xv6-riscv/user

```c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                    // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                        // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                // step 2: become echo
    printf("exec failed\n");           // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                       // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Figure 5.1: run.c

```
131
132 UPROGS=\
133         $U/_cat\
134         $U/_echo\
135         $U/_sleep\
136         $U/_args\
137         $U/_mystat\
138         $U/_find\
139         $U/_run\
140         $U/_forktest\
141         $U/_grep\
142         $U/_init\
143         $U/_kill\
```

Figure 5.2: run.c inside UPROGS

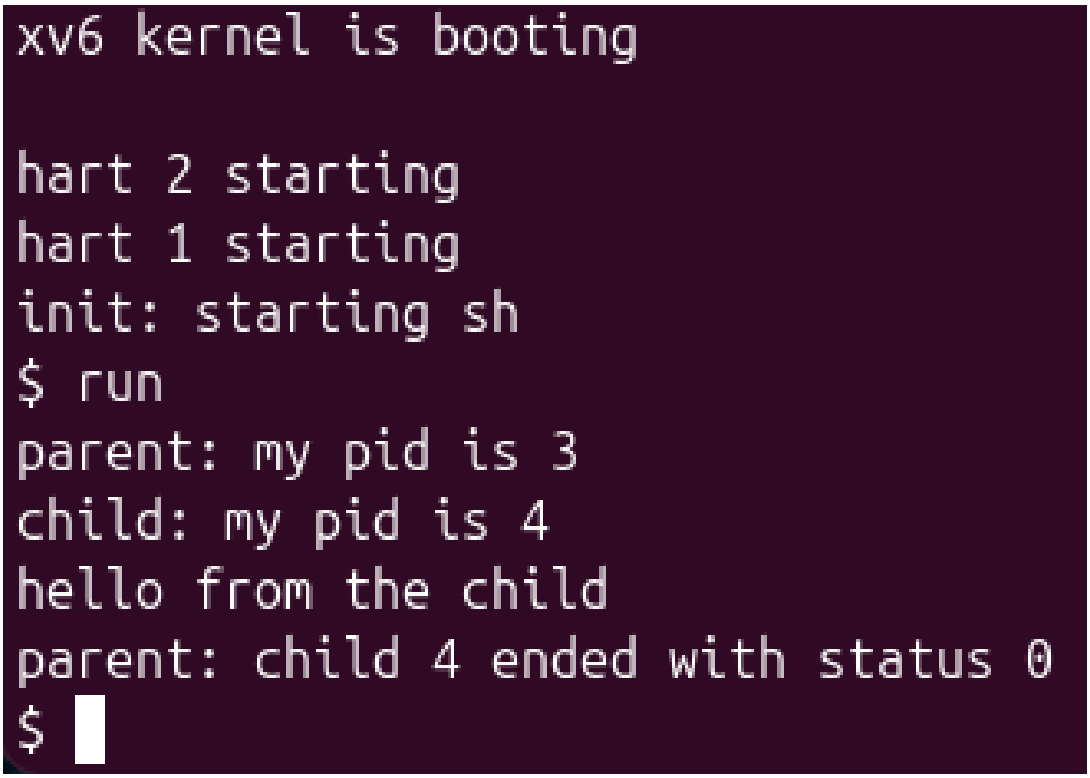


Figure 5.3: Results of run.c

```
char *args[] = { "ls", 0 };
exec("ls", args);  // step 2: become ls
```

```
if(pid == 0){                          // only the child runs this
  printf("child: my pid is %d\n", getpid());
  char *args[] = { "ls", 0 };
  exec("ls", args);                  // step 2: become ls
  printf("exec failed\n");             // runs only if exec failed
  exit(1);
}
```

Figure 5.4: run.c changes

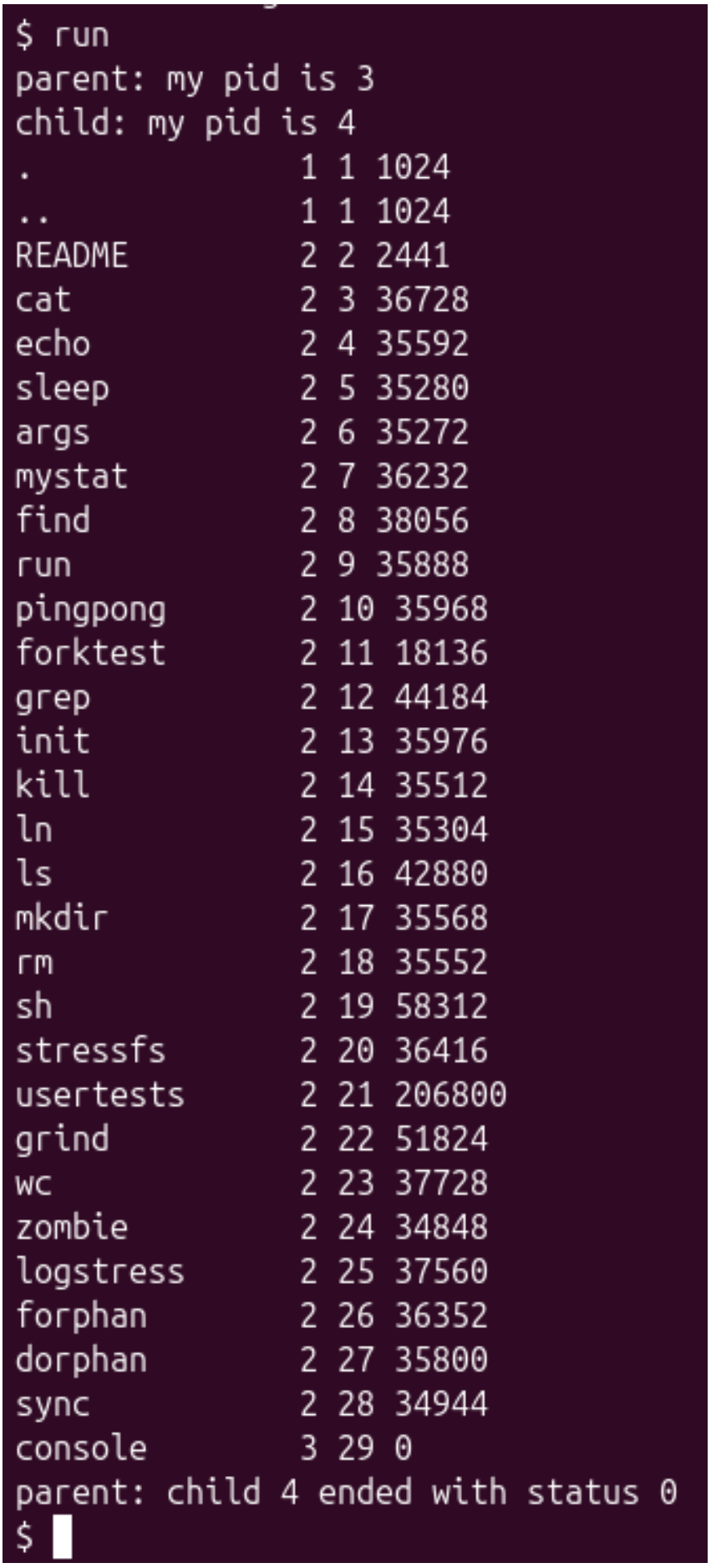


Figure 5.5: Results of modified run.c

```
1 exec("nosuchprog", args);
```

```
if(pid == 0){                          // only the child runs this
  printf("child: my pid is %d\n", getpid());
  char *args[] = { "ls", 0 };
  exec("nosuchprog", args);            // step 2: program that doesn't exist
  printf("exec failed\n");             // runs only if exec failed
  exit(1);
}
```

Figure 5.6: run.c changes

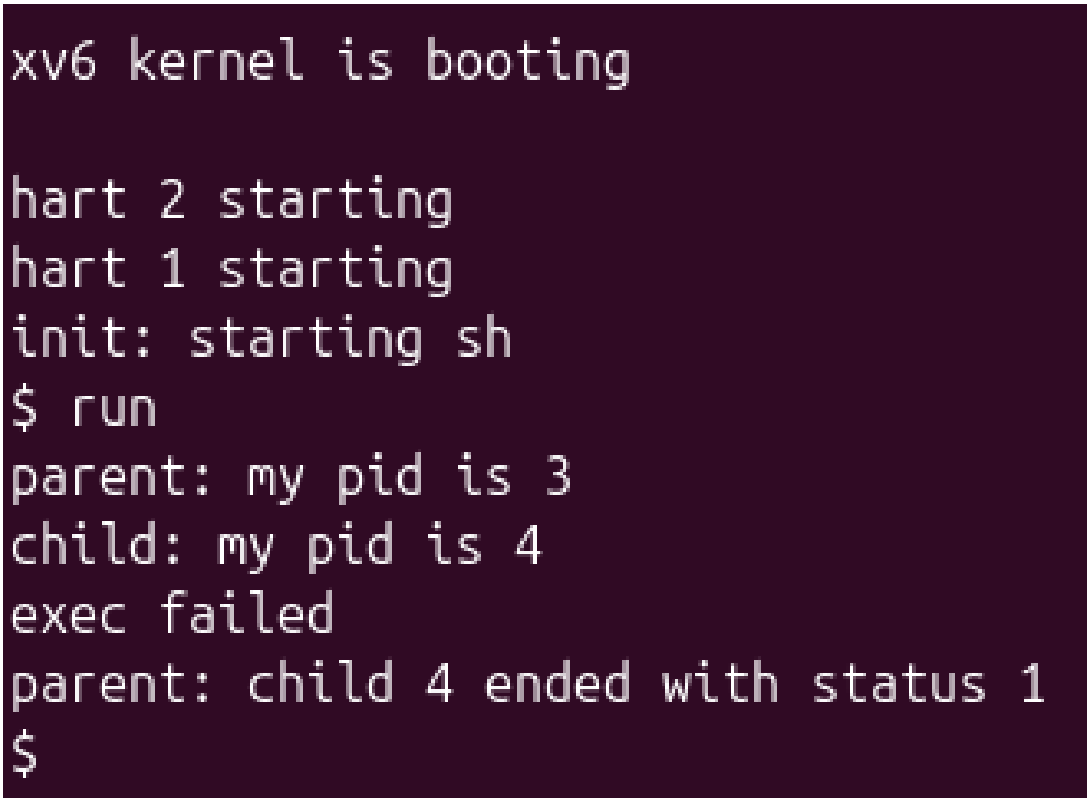


Figure 5.7: Results of invalid run.c

1. **Write the three calls that `run.c` uses in xv6 and the three calls `run.c` used on Linux in Lab 7. What is the same?**
   Linux: `fork`, `execlp`, `waitpid`. xv6: `fork`, `exec`, `wait`.
   The three steps are the same: `fork` copies the process, `exec` replaces the child with another program, and `wait` makes the parent wait for the child. The differences are only in the details: `execlp` takes a list of arguments and searches `$PATH`, while xv6's `exec` takes a file name and an array of arguments and does not search any path; `waitpid` with `WEXITSTATUS` on Linux versus `wait(&status)` in xv6.

2. **Who prints "hello from the child": the code of run, or echo?**
   `echo` prints it. After `exec`, the child is no longer the code of `run`; it has become `echo`, which prints its arguments to descriptor 1.

3. **Why is the status 1 here and 127 on Linux? (Look at the exit line after `exec` in both programs.)**
   In both cases `exec` fails because `nosuchprog` does not exist, so the child reaches the exit line after it. The status is just the number we wrote there: `exit(1)` in the xv6 program and `exit(127)` in the Linux program, where we chose 127 to imitate the shell's ``` ``command not found'' ``` code.

Part 2. pingpong

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    // TODO: read 1 byte from p2c[0] into b
    printf("%d: received ping\n", getpid());
    // TODO: write the byte b into c2p[1]
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  // TODO: write 1 byte into p2c[1]
  // TODO: read 1 byte from c2p[0] into b
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

Open pingpong.c ~/xv6-riscv/user Sa

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);                // block until the parent's byte arrives
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);               // send the same byte back
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  b = 'x';
  write(p2c[1], &b, 1);                 // send one byte to the child
  read(c2p[0], &b, 1);                  // block until the child replies
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
  exit(0);
}

```

Figure 6.1: pingpong.c

```
132 UPROGS=\
133         $U/_cat\
134         $U/_echo\
135         $U/_sleep\
136         $U/_args\
137         $U/_mystat\
138         $U/_find\
139         $U/_run\
140         $U/_pingpong\
141         $U/_forktest\
142         $U/_grep\
143         $U/_init\
144         $U/_kill\
145         $U/_ln\
146         $U/_ls\
```

Figure 6.2: pingpong.c inside UPROGS

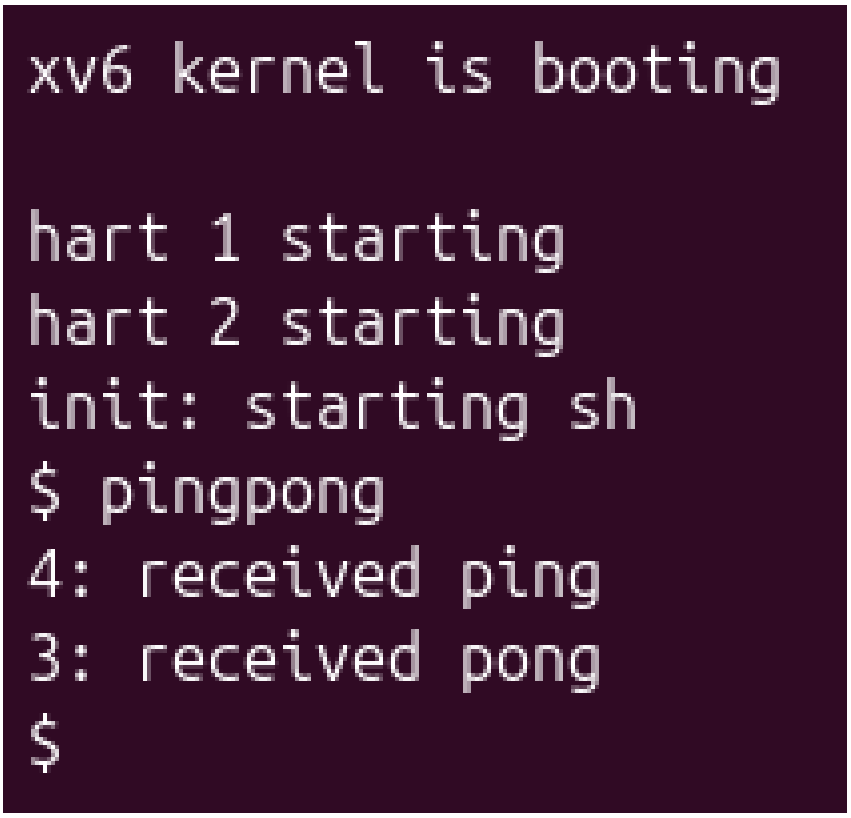


Figure 6.3: Results of pingpong.c

1. **Why do you need TWO pipes, and not one for both directions?**
   A pipe is one-directional, so each direction needs its own pipe. With a single pipe, the parent could read back the byte it has just written itself before the child gets it, and the child would then wait forever. Using `p2c` (parent to child) and `c2p` (child to parent) means each process only reads what the other one wrote.

2. **Nothing in your code says "wait until the child has replied". Which call makes the parent wait?**
   The `read(c2p[0], &b, 1)` on line 23. The pipe is empty until the child writes its byte (line 16), and `read` on an empty pipe blocks the parent until a byte arrives. (Line 14 is the same mechanism, but it makes the *child* wait for the parent's byte.)

CONCLUSION

In these labs I ran, modified and broke small C programs that do what a shell does behind the scenes. I saw that a new program is started in three steps: `fork` copies the process, `exec` replaces the copy with another program, and `wait` lets the parent collect the exit status. The code after `exec` runs only if `exec` fails, which is why `perror` appeared only with `nosuchprog`, and the status 127 (Linux) and 1 (xv6) are simply the numbers written in our own `exit` calls. With the zombie experiment I saw that a terminated child stays in the process table, marked `Z`, until its parent calls `waitpid`, which removes it.

I also learned that redirection and pipes are based on file descriptors. Since `open` and `dup` always return the smallest free descriptor, closing descriptor 1 before opening `out.txt` makes `ls` write into the file, even though `ls` itself does not change. In the pipeline, `ls /etc | wc -l` and my program gave the same result (238). When I commented out `close(p[1])` in the parent, `wc` never ended, because end-of-file arrives only when all copies of the write end are closed. This showed me that forgetting to close one descriptor can block a whole program.

In xv6 the same three calls work in the same way, with small differences: `exec` takes an array of arguments and does not search a `PATH`, and `wait` returns the status directly. Finally, `pingpong` showed that pipes are one-directional, so two pipes are needed for communication in both directions, and that a blocking `read` on an empty pipe is what makes the parent wait for the child's reply. In conclusion, the labs showed that processes, redirection and pipes are built from a few simple system calls, and that the shell is an ordinary program that combines them.

REFERENCES

1. https://github.com/mit-pdos

2. https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf

3. www.geeksforgeeks.org/c/exec-family-of-functions-in-c/

4. https://else.fcim.utm.md/course/section.php?id=19066